

edit time: 2026-05-28 19:08:41 RL基础与迭代算法
有点意思。随着“期望累积折扣奖励”八个字喊多了，慢慢对这一块公式也熟悉起来了。
edit time: 2026-05-29 16:10:17 工程优化与收敛性理论

目录

- (RL基础与迭代算法)
- MDP、Bellman期望方程、Bellman最优方程
- 值迭代、策略迭代
- Monte Carlo的prediction和control
- RL Regret和UCB-VI
- TD学习的prediction：TD(0) 和 control：SARSA、Q-learning
- (工程优化与收敛性理论)
- on-policy/off-policy、online/offline、工程优化
- model-based/model-free、收敛性理论
- TD(λ)、LSTD、平均奖励MDP

	Prediction (评估 π)	Control (找 π^*)
DP (要模型)	策略评估	值迭代 / 策略迭代
MC (无模型、不 bootstrap)	MC 评估	MC control + ϵ -greedy
TD on-policy	TD(0) on V	SARSA
TD off-policy	重要性采样 TD	Q-learning

算法	收敛性	条件	速度/界
Planning (已知模型)			
值迭代	全局	$\gamma < 1$	$O(\gamma^k)$
策略迭代	全局	有限 MDP	有限步
表格 RL			
Q-learning	全局 w.p.1	无限访问	$O(1/k)$
SARSA	全局 w.p.1	GLIE	$O(1/k)$
TD(λ)	全局	On-policy	依赖 λ
UCB-VI	Regret 界	Episodic	$\tilde{O}(\sqrt{H^3 S A K})$
线性函数逼近			
Linear TD	收敛	On-policy	放大 $\frac{1}{\sqrt{1-\gamma^2}}$
Linear Q	可能发散	Off-policy	—
GTD2	收敛	Off-policy	$O(1/k)$
LSTD	闭式解	批量	$O(\sqrt{d/n})$
Multi-Agent			
Minimax-Q	纳什均衡	两人零和	$O(\gamma^k)$
独立 Q	可能不收敛	一般和	—

MDP、Bellman期望方程、Bellman最优方程

MDP是一个五元组 (S, A, P, R, γ) 。

记住以下两个定义：

值函数 $V^\pi(s)$ ：从状态 s 出发，按策略 π 行动。所以是**期望累积折扣奖励**

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right]$$

你还真别说，要硬想不如直接从左往右记期望累积折扣奖励这八个字。

动作-值函数 $Q^\pi(s, a)$ ：从状态 s 采取动作 a ，然后按 π 行动。所以是**即时奖励加折扣未来期望价值**

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^\pi(s')$$

以后看到期望累积折扣奖励和即时奖励加折扣未来期望价值这两个形式，一定要整体地看。

eg1. 动作-值函数代入具体值

设状态 s ，动作 a ，奖励 $R(s, a) = 1$ ， $\gamma = 0.9$ ，有50%概率到 s_1 且 $V^\pi(s_1) = 5$ ，另外50%概率到 s_2 且 $V^\pi(s_2) = 3$ 。求 $Q^\pi(s, a)$ 。

解： $Q^\pi(s, a)$ 是即时奖励加上折扣未来期望价值。

$$Q^\pi(s, a) = 1 + 0.9 \times (0.5 \times 5 + 0.5 \times 3) = 4.6$$

Bellman方程是值函数的递推关系。

Bellman期望方程往往用于评估当前状态的好坏（prediction，用到 $V^\pi(s)$ 的递推关系，直觉上prediction主要针对 s ），

Bellman最优方程往往用于做出下一个动作（control，用到 $Q^*(s, a)$ 的递推关系，直觉上control主要针对 (s, a) ）。

Bellman期望方程：值函数是动作-值函数对动作的期望。

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a)$$

也可以把动作-值函数展开为值函数，得到只有值函数的递推的另一种形式：

$$V^\pi(s) = \sum_a \pi(a|s) [R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s')]$$

当然也可以写成期望的形式，内层期望是对可转移到的状态 s' 的，外层期望是对动作 a 的：

$$V^\pi(s) = \mathbb{E}_{a \sim \pi} [R(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} [V^\pi(s')]]$$

eg2. 2状态MDP

下述是一个无动作、环境自动转移（直接去掉上式的 π 和 a ）的例子。

- 学习中 $\rightarrow$ 60% 继续学习，40% 开始摸鱼
- 摸鱼中 $\rightarrow$ 30% 开始学习，70% 继续摸鱼

奖励：学习+2 分，摸鱼+1 分。 $\gamma = 0.9$

则我们可以列出以下的Bellman期望方程：

$$V(\text{学习}) = 2 + 0.9 \times (0.6V(\text{学习}) + 0.4V(\text{摸鱼}))$$

$$V(\text{摸鱼}) = 1 + 0.9 \times (0.3V(\text{学习}) + 0.7V(\text{摸鱼}))$$

解得 $V(\text{学习}) = 14.85$, $V(\text{摸鱼}) = 13.54$ 。

eg3. Bellman期望方程的证明

首先根据期望的可加性，可以把第一项拆出来

$$\begin{aligned}
V^\pi(s) &= \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right] \\
&= \mathbb{E}_\pi [R(s_0, a_0) \mid s_0 = s] + \gamma \mathbb{E}_\pi \left[\sum_{t=1}^{\infty} \gamma^{t-1} R(s_t, a_t) \mid s_0 = s \right]
\end{aligned}$$

第一项用期望的定义按 a 拆开：

$$\mathbb{E}_\pi [R(s_0, a_0) \mid s_0 = s] = \sum_a \pi(a|s) R(s, a)$$

再处理第二项，记 $G_1 = \sum_{t=1}^{\infty} \gamma^{t-1} R(s_t, a_t)$ ，第二项即 $\gamma \mathbb{E}_\pi [G_1 \mid s_0 = s]$ 。

我们期望把第二项配凑成值函数那样条件是 s' 的形式，但是现在的条件是 s 。因此使用全期望公式（相当于取划分变量），依次展开 a_0 和 s_1 ，把期望拆成条件期望之和：

$$\mathbb{E}_\pi [G_1 \mid s_0 = s] = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) \mathbb{E}_\pi [G_1 \mid s_0 = s, a_0 = a, s_1 = s']$$

现在就好说了，根据马尔科夫性，给定 $s_1 = s'$ 之后，未来的回报与 s_0, a_0 无关。

所以内层期望应该简化为 $\mathbb{E}_\pi [G_1 \mid s_1 = s']$ ，这正是从状态 s' 出发，按策略 π 行动的值函数。

故有：

$$\mathbb{E}_\pi [G_1 \mid s_0 = s] = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) V^\pi(s')$$

接下来把第一项与第二项合并，提出 $\sum_a \pi(a|s)$ ，就得到了Bellman期望方程：

$$V^\pi(s) = \sum_a \pi(a|s) [R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s')]$$

接下来给出Bellman最优方程。

首先定义最优值函数，在各个不同的 π 的值函数/动作-值函数中，取 $\max_\pi$ 。

$$V^*(s) = \max_\pi V^\pi(s)$$

$$Q^*(s, a) = \max_\pi Q^\pi(s, a)$$

上述两式的关联是，在最优情况下有

$$V^*(s) = \max_a Q^*(s, a)$$

最优值函数就是最优动作-值函数对动作取max。

即从状态 s 出发，能获得的最大期望回报，就是选择最优动作对应的 Q 值。

“每一次都选择当前最优的那个动作”

Bellman最优方程：

动作-值函数可以换成最优动作-值函数得到Bellman最优方程的形式。

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s')$$

(当然也可以把上式的 $V^*(s')$ 换成 $\max_a Q^*(s', a)$ 得到纯粹 Q 的递推，在此省略。)

根据最优动作-值函数和值函数的关联：

$$V^*(s) = \max_a Q^*(s, a) = \max_a [R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s')]$$

顺便提一下，最优策略就是最优值函数对应的策略，即最优动作-值函数对动作取argmax：

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a) = \operatorname{argmax}_a [R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s')]$$

ps. 严谨来说，最优策略不一定是确定性的，但在 MDP 中，存在一个确定性的最优策略。

eg4. 最优策略代入具体值

- 动作A：食堂（确定7分满意度）
- 动作B：新餐厅（60% 概率9分超好吃，40% 概率5分踩雷）

比较两个动作。

我们可以比较两个动作-值函数：

$$Q^*(s, \text{食堂}) = 7$$

$$Q^*(s, \text{新餐厅}) = 0.6 \times 9 + 0.4 \times 5 = 7.4$$

故最优策略是总是选新餐厅， $V^*(s) = Q^*(s, \text{新餐厅}) = 7.4$ 。

值迭代、策略迭代

之前玩味了一些RL的定义（其实这一块就是定义一开始最费劲），接下来看看迭代。

Bellman期望方程也可以写作算子形式：

$$V^\pi = T^\pi V^\pi$$

即 V^π 是Bellman算子 T^π 的不动点。

由于我们引入了 γ ，使得**Bellman最优算子是 γ -收缩映射**：

$$\|TV - TV'\|_{\infty} \leq \gamma \|V - V'\|_{\infty}$$

证明如下，主要使用了绝对值的性质：

$$(T^*V)(s) - (T^*V')(s) \leq [R(s, a^*)] + \gamma \sum_{s'} P(s'|s, a^*) V(s') - [R(s, a^*)] + \gamma \sum_{s'} P(s'|s, a^*) V'(s')$$

$$(T^*V)(s) - (T^*V')(s) \leq \gamma \sum_{s'} P(s'|s, a^*) [V(s') - V'(s')]$$

$$(T^*V)(s) - (T^*V')(s) \leq \gamma \left| \sum_{s'} P(s'|s, a^*) [V(s') - V'(s')] \right|$$

求和的绝对值小于等于绝对值的求和，然后 P 一定非负，则有

$$(T^*V)(s) - (T^*V')(s) \leq \gamma \sum_{s'} P(s'|s, a^*) |V(s') - V'(s')|$$

对每一项 s' 取上界并提出来，即根据 $|V(s') - V'(s')| \leq \max_{s'} |V(s') - V'(s')|$ 有

$$(T^*V)(s) - (T^*V')(s) \leq \gamma \left(\sum_{s'} P(s'|s, a^*) \right) \max_{s'} |V(s') - V'(s')|$$

概率求和为 1，故有

$$(T^*V)(s) - (T^*V')(s) \leq \gamma \max_{s'} |V(s') - V'(s')|$$

$\max$ 也可以写为无穷范数，故有下式得证

$$(T^*V)(s) - (T^*V')(s) \leq \gamma \|V - V'\|_{\infty}$$

同理也有

$$(T^*V')(s) - (T^*V)(s) \leq \gamma \|V - V'\|_{\infty}$$

故左边二者的 $\max$ （a.k.a 无穷范数）小于等于右式，故Bellman算子是收缩映射得证

$$\|T^*V - T^*V'\|_{\infty} \leq \gamma \|V - V'\|_{\infty}$$

Banach不动点定理：若 T 是完备度量空间上的 γ -收缩映射（ $\gamma < 1$ ），则 T 有唯一不动点 V^* ，从任意 V_0 出发时有 $V_k = T^k V_0 \rightarrow V^*$ ，收敛速度 $\|V_k - V^*\|_{\infty} \leq \gamma^k \|V_0 - V^*\|_{\infty}$ 。这保证了值迭代算法一定能到达最优不动点，且迭代速度是 γ^k 程度的。

值迭代算法就是任意地初始化值函数（置为0），然后通过压缩映射使其迭代到最优值函数，之后再输出最优值函数对应的最优策略。

Algorithm 1 值迭代 (Value Iteration)

```
1: 初始化  $V_0(s) = 0$  对所有  $s$ 
2: for  $k = 0, 1, 2, \dots$  do
3:   for 每个状态  $s$  do
4:      $V_{k+1}(s) = \max_a [R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s')]$ 
5:   end for
6:   if  $\|V_{k+1} - V_k\|_\infty < \epsilon(1 - \gamma)/(2\gamma)$  then
7:     停止
8:   end if
9: end for
10: 输出  $\pi(s) = \arg \max_a [R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s')]$ 
```

观察一下值迭代的更新公式：

$$V_{k+1}(s) = \max_a [R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s')]$$

它是把 $V_k(s)$ 更新为当前的Bellman最优方程求得的值，得到 $V_{k+1}(s)$ 。

策略迭代算法则是更进一步，

更激进地每次都跳到当前策略下的最优（需要每次都算出完整的 V^π ）。

$$\pi'(s) = \operatorname{argmax}_a Q^\pi(s, a) = \operatorname{argmax}_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right]$$

注意一下区别：值迭代时“只走一步”，有了 V_k 就更新到 V_{k+1} 了；

而策略迭代需要在评估时算出 V^π ： V^π 和 V_k 区别就很大了。

我们在策略迭代公式会看到策略迭代没有 k 这样的步骤标记。根据我们对值函数的定义的了解（期望累积折扣奖励），求出 V^π 时一定已经隐式考虑了未来的所有时间步，而不是像值迭代那样只往前看一步，尽管值迭代的多轮迭代也会逐步把未来信息传播回来。

策略迭代最多 $|A|^{|S|}$ 次迭代收敛到最优策略，实际迭代很快。

Monte Carlo

环境的动态特性（状态转移概率 P 和奖励函数 R ）不依赖于具体的时间步 t ，只依赖于当前状态和动作。因此从任意的时间步 t 开始，都有

$$V^\pi(s) = \mathbb{E}_\pi(G_t | s_t = s)$$

其中 G_t 是累积折扣奖励， $V^\pi(s)$ 作为期望累积折扣奖励即它的条件期望：

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

特别地， $V^\pi(s) = \mathbb{E}_\pi(G_0 | s_0 = s)$ 成立。

注意值函数输入的“参数”是 s ，传入作为初始状态，不要在条件里不小心把 s 和 s_t 写反了。

联系一下：

bandit不知道每个老虎机的奖励期望 μ_a 是多少，所以用样本均值 $\hat{\mu}_a = \frac{1}{N_a} \sum_{i=1}^{N_a} r_i$ 估计 μ_a 。

MDP不知道值函数 $V^\pi(s)$ （累积折扣奖励 G_t 的期望）是多少，所以用样本均值

$\hat{V}(s) = \frac{1}{N_s} \sum_i G^{(i)}$ 估计 $V^\pi(s)$ 。

总结1: **model-free prediction**需要用样本均值估计值函数 V 。

我们可以把下述样本均值写成迭代的形式

$$V_n(s) = \frac{1}{n} \sum_{i=1}^n G^{(i)}$$

等价于

$$V(s) \leftarrow V(s) + \frac{1}{N(s)} [G_t - V(s)]$$

当然，上述是严格平均式子，每次除以访问次数，最终收敛到精确均值。也可以改成常数步长形式 α ，用于处理一些非平稳的情况：

$$V(s) \leftarrow V(s) + \alpha [G_t - V(s)]$$

使用上述的除以访问次数的式子，有了Monte Carlo评估算法（MC评估）。

定义 6.1 (首次访问 MC 评估). 给定策略 π 。运行 K 条完整轨迹：

Algorithm 2 First-Visit Monte Carlo 评估

```
1: 初始化  $V(s) = 0$ , 访问计数  $N(s) = 0$ 
2: for episode  $k = 1, \dots, K$  do
3:   用  $\pi$  生成完整轨迹  $(s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T)$ 
4:   计算  $G_t = \sum_{j=t}^{T-1} \gamma^{j-t} r_j$  对每个  $t$ 
5:   for 每个在该轨迹中首次出现的状态  $s$ , 对应时刻  $t$  do
6:      $N(s) += 1$ 
7:      $V(s) \leftarrow V(s) + \frac{1}{N(s)} [G_t - V(s)]$ 
8:   end for
9: end for
```

First-visit MC：一条轨迹里s出现多次只用第一次的G。样本独立，方差分析干净。

Every-visit MC：s 每次出现都更新。样本量更大但相关。

两者都以概率1 (w.p.1) 收敛到 V^π 。第一种严格无偏，第二种渐近无偏。

实践用every-visit 的多（数据利用率高）。

考虑单次 G_t 的方差。假设每步奖励方差为 σ^2 ，乐观考虑各步独立，则有

$$\text{Var}(G_t) = \sum_{k=0}^{\infty} \gamma^{2k} \sigma^2 = \frac{\sigma^2}{1 - \gamma^2}$$

（如果 $\gamma = 0.99$ ，则 G_t 会把单步方差放大约50倍）

样本均值的标准差是 $\sqrt{\text{Var}(G_t)/n}$ ，则要达到精度 ϵ 需要的轨迹数量为：

$$n = O\left(\frac{\sigma^2}{(1 - \gamma^2)\epsilon^2}\right)$$

实际情形（状态转移随机性叠加奖励随机性）方差更大，使得MC需要采样很多次轨迹用于评估，才能得到一个比较好的估计。

上述解决了评估（prediction）问题。那么控制（找最优的策略 π ）怎么搞？

考虑到之前我们提到的最优策略

$$\pi'(s) = \operatorname{argmax}_a Q^\pi(s, a)$$

我们如果像评估一样估计 $V^\pi(s)$ ，则要找到最优策略需要用 $Q^\pi(s, a)$ 的定义式转换一下，需要引入额外的 P 和 R ，不好。因此控制时不如直接估计 Q ，从而找最优策略 $\pi'(s)$ 。
总结2: **model-free control需要直接估计 Q ，不能只估计 V 。**

Algorithm 3 MC Control (ϵ -greedy 版)

```

1: 初始化  $Q(s, a)$  任意,  $\pi = \epsilon$ -greedy w.r.t.  $Q$ 
2: for episode  $k = 1, 2, \dots$  do
3:   用  $\pi$  生成完整轨迹
4:   对每个访问到的  $(s, a)$ :  $Q(s, a) \leftarrow Q(s, a) + \alpha[G - Q(s, a)]$ 
5:    $\pi \leftarrow \epsilon$ -greedy w.r.t. 更新后的  $Q$ 
6: end for

```

和MC prediction提到的算法相比，这里使用了常数步长 α 用于适应复杂和非平稳的环境，并且使用了 ϵ -greedy。

后者是因为，纯greedy 会漏掉好动作——没访问过的 (s, a) 永远没有 Q 估计、永远不会被选中、永远没机会被试。 ϵ -greedy 保证每个动作都有 $\geq \epsilon/|A|$ 的概率被尝试，对应bandit 的 forced exploration。

在GLIE 条件（Greedy in the Limit with Infinite Exploration，即 $\epsilon \rightarrow 0$ 、每个 (s, a) 被无限次访问）下，MC control w.p.1 收敛到 Q^* 。

MC有两个问题，都集中在“样本”。

一是必须等到episode结束才能计算 G_t ，不适用于无限horizon任务和在线学习（引入后续方法：时序差分，用bootstrap，导出SARSA和Q-learning）；

二是方差很大， G_t 把各步噪声叠加起来，需要承担一整段的随机性，难以设置学习率。

（引入后续方法：UCB-VI估计 $\hat{P}$ 和 $\hat{R}$ ，model-based RL）

RL Regret和UCB-VI

Episodic MDP的Regret定义：运行 K 个episode，每个episode长度为 H ，则有Regret是这 K 个episode的Regret的总和

$$\text{Regret}(K) = \sum_{k=1}^K [V_1^*(s_1^k) - V_1^{\pi_k}(s_1^k)]$$

其中 π_k 是第 k 个episode使用的策略。

这里的下标1比较神秘，或许是 $h = 1$ 处的值函数和状态的意思。

维度	Bandit	MDP
单轮损失	$\mu^* - \mu_a$	$V^*(s) - V^\pi(s)$
累积	T 轮	K 个 episode
最优界	$O(\sqrt{KT})$	$O(\sqrt{H^3 SAK})$

Algorithm 4 UCB-VI (完整版)

```

1: 初始化计数  $N(s, a) = 0, N(s, a, s') = 0$ 
2: for episode  $k = 1, 2, \dots, K$  do
3:   // 乐观值迭代 (从后往前)
4:    $V_{H+1}(s) = 0$  for all  $s$ 
5:   for  $h = H, H-1, \dots, 1$  do
6:     for each  $(s, a)$  do
7:       if  $N(s, a) > 0$  then
8:          $\hat{P}(s'|s, a) = N(s, a, s')/N(s, a)$ 
9:          $\text{bonus}_h(s, a) = c\sqrt{\frac{H^2 \iota}{N(s, a)}}$  where  $\iota = \ln(SAHK/\delta)$ 
10:      else
11:         $\text{bonus}_h(s, a) = H$  ▷ 最大可能值
12:      end if
13:       $Q_h(s, a) = \min \left\{ H, R(s, a) + \sum_{s'} \hat{P}(s'|s, a) V_{h+1}(s') + \text{bonus}_h(s, a) \right\}$ 
14:    end for
15:     $V_h(s) = \max_a Q_h(s, a), \quad \pi_h^k(s) = \arg \max_a Q_h(s, a)$ 
16:  end for
17:  // 执行策略并收集数据
18:  从  $s_1^k$  出发, 执行  $\pi^k$ , 观察轨迹  $(s_1, a_1, r_1, \dots, s_H, a_H, r_H)$ 
19:  更新计数:  $N(s_h, a_h) += 1, N(s_h, a_h, s_{h+1}) += 1$ 
20: end for

```

可以看到，UCB-VI首先估出了 $\hat{P}$ 和 $\hat{R}$ (?)，再做planning。

UCB-VI的策略称为“乐观greedy”。

UCB-VI的Regret上界：以概率至少 $1 - \delta$ ，有

$$\text{Regret}(K) = \sum_{k=1}^K \left[V_1^*(s_1^k) - V_1^{\pi^k}(s_1^k) \right] \leq O\left(\sqrt{H^3 SAK\iota}\right),$$

其中

$$\iota = \ln(SAHK/\delta).$$

证明略去。证明细节不算这周的主干，UCB-VI方法也在本章出现得比其他大明星方法少。有兴趣可以查阅原文。

信息论Regret下界：存在 MDP 使得任何算法的期望 Regret 至少：

$$\mathbb{E}[\text{Regret}(K)] \geq \Omega(\sqrt{HSAK})$$

时序差分 (TD) 学习, SARSA, Q-learning

MC的问题在于从 s_t 出发，需要把整段轨迹跑完才能计算 G_t 从而更新 $V(s_t)$ 。

时序差分的想法是，能不能不把整段轨迹跑完，只使用部分的轨迹估计 G_t 用于更新 $V(s_t)$ ？

考虑到

$$G_t = r_t + \gamma G_{t+1}$$

上述这个级数关系推出的递归式子是一切的起源。

如果有估计 $V(s_{t+1}) = \mathbb{E}[G_{t+1}|s_{t+1}]$ ，则可以用它代替未知的 G_{t+1} ：

$$G_t = r_t + \gamma V(s_{t+1})$$

右边的 $r_t + \gamma V(s_{t+1})$ 称为TD target。

这样一步就能更新（观察到 (s_t, a_t, r_t, s_{t+1}) 四项就行，如果这里的“一步”更新暂时没太看懂，可以看下述的eg5），而不用等到episode结束。

这样用自己的估计替换真实值的操作称为**bootstrap**（自举）。

“自举：用估计值构造目标”

TD(0) prediction算法：观察到 (s_t, a_t, r_t, s_{t+1}) 后，

$$V(s_t) \leftarrow V(s_t) + \alpha [r_t + \gamma V(s_{t+1}) - V(s_t)]$$

（其中TD误差定义为 $\delta_t := r_t + \gamma V(s_{t+1}) - V(s_t)$ ，可以写作 $V(s_t) \leftarrow V(s_t) + \alpha \delta_t$ ）

（注意把TD(0) prediction与MC的 $V(s) \leftarrow V(s) + \alpha [G_t - V(s)]$ 进行比较，发现只是把采样整个episode才能拿到的 G_t ，换成了对下一步的预测值 $r_t + \gamma V(s_{t+1})$ 而已）

一定要注意，这里是用预测 $t+1$ 处的 $V(s_{t+1})$ 反过来更新预测 t 处的 $V(s_t)$ 。

“先把后面的步骤推导出来，然后回过头去检查并修正你第一步的计算。这不叫时光倒流，这叫根据新信息进行逻辑推理和修正：用后一步的估计值，来更新前一步的估计值。”

ps. 对衰减因子 γ 在这里的作用简单说明一下。

括号里的 $r_t + \gamma V(s_{t+1})$ 是对 DP 中 $R(s, a) + \gamma \sum s' P(s'|s, a) V(s')$ 的单样本估计。

当 γ 越大时，

$V(s_{t+1})$ 的权重越大，意味着 TD 更依赖 long-term 的价值估计，这正好是 DP 递归结构的核心，所以得到一个对 DP 解更好的估计。

与此同时，相比于 immediate rewards，对 later rewards 考虑得越多。

ps2. TD(0) 的收敛性定理移到了下述收敛性理论部分。

eg5. MC vs TD

给定轨迹： $s_0 \rightarrow (r_0 = 1) \rightarrow s_1 \rightarrow (r_1 = 2) \rightarrow s_2 \rightarrow (r_2 = 4) \rightarrow \text{终止}$ ， $\gamma = 1$ 。

已有当前估计： $V(s_0) = 10, V(s_1) = 5, V(s_2) = 3$ 。取步长 $\alpha = 0.1$ 。

请分别给出 MC 和 TD 对 $V(s_0)$ 的更新。

解：

(MC)

$$G_0 = r_0 + \gamma r_1 + \gamma^2 r_2 = 1 + 2 + 4 = 7$$

$$V(s_0) \leftarrow 10 + 0.1 \times (7 - 10) = 9.7$$

(TD)

$$\text{TD target} = r_0 + \gamma V(s_1) = 1 + 5 = 6$$

$$V(s_0) \leftarrow 10 + 0.1 \times (6 - 10) = 9.6$$

注意上述其余部分没有任何区别，仅仅只有两者使用的更新目标不同：

一个使用了 G_0 （需要完整 episode；真实，带有 r_0, r_1, r_2 三者的方差），

而另一个使用了 $\text{TD target} = r_0 + \gamma V(s_1)$ （拿到 s_1 时就可以立刻发生，不需要等完整 episode 结束；取决于 $V(s_1)$ 估得准不准，只承担 r_0 一步的噪声）。

从上述说明也可以看到，TD 比 MC 的方差要显著降低，因为它只承担一步奖励 r 的随机性。

代价是，如果 $V(s_{t+1})$ 有偏差 b ，那么 TD target 就会有偏差 γb ，存在偏差-方差权衡，所以有后续的 TD(λ) 方法。

接下来把 TD(0) 从 V 推广到 Q ，需要的样本变为五元组 $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$ 。

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

增加的下一步动作 a_{t+1} 从哪里来，决定了得到的是on-policy (SARSA) 还是off-policy算法 (Q-learning)。

SARSA算法：名字超级形象，来自更新用的五元组 $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$ 。

SARSA算法有：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

其中 a_{t+1} 是在 s_{t+1} 用 ϵ -greedy选择的。

Algorithm 5 SARSA

```
1: 初始化  $Q(s, a)$  任意
2: 在  $s_0$  用  $\epsilon$ -greedy (基于当前  $Q$ ) 选  $a_0$ 
3: for 每步  $t$  do
4:   执行  $a_t$ , 观察  $r_t, s_{t+1}$ 
5:   在  $s_{t+1}$  用  $\epsilon$ -greedy 选  $a_{t+1}$ 
6:    $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$ 
7:    $t \leftarrow t + 1$ 
8: end for
```

关键： a_{t+1} 是即将真正执行的下一步动作，按照当前behavior policy (ϵ -greedy) 采样。

SARSA是**on-policy**的，它的TD target里的 $a_{t+1} \sim \pi_{behavior}$ ，产生数据的策略和被评估的策略是同一个。

所以SARSA学到的 Q 是当前 ϵ -greedy策略本身的 Q^π ($\pi = \epsilon - greedy$ w.r.t. Q , w.r.t.是 with respect to, 关于)。

如果要使这个 $Q \rightarrow Q^*$ ，必须满足GLIE条件 (Greedy in the Limit with Infinite Exploration)，例如 $\epsilon_t = 1/t$ ：

- 长期 $\epsilon \rightarrow 0$ ，使得behavior policy趋向于greedy，
- 但 ϵ 下降不能太快，否则GLIE探索不充分。

Q-learning算法：把 $Q(s_{t+1}, a_{t+1})$ 换成 $\max_{a'} Q(s_{t+1}, a')$ 。

Q-learning算法有：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

关键：

$$\pi_{greedy}(s) = \operatorname{argmax}_a Q(s, a)$$

这里的target用于评估的是一个不同的策略，称为greedy（“无探索的纯贪婪”），但智能体实际行动时仍用 ϵ -greedy（必须有探索才能更新所有 (s, a) ）。greedy和 ϵ -greedy不同，称Q-learning是**off-policy**的。

对比SARSA和Q-learning：
这样的话，Q-learning的评估与学习学到的是最优策略对应的 Q^* ，与使用什么探索策略无关（前提是所有 (s, a) 都能被访问到）。相比起来，SARSA要学 Q^* 需要让 $\epsilon \rightarrow 0$ ，而Q-learning直接学 Q^* ， ϵ 只影响样本质量而不影响目标，形成Q-learning相比于SARSA的优势。
“Q-learning 假装自己最终会完美，SARSA 承认自己永远会犯小错”
代价是，Q-learning有Deadly Triad（“死亡三角”，三个影响稳定性的因素）。

on/off-policy、online/offline、工程优化

算法	Model	On/off	On/offline	关键技巧
值迭代	based	—	不交互	直接用 P, R
策略迭代	based	—	不交互	评估 + 改进交替
MC 评估	free	on	online	整段 G_t 取均值
MC control	free	on	online	ϵ -greedy
UCB-VI	based	on	online	估模型 + 乐观 bonus
SARSA	free	on	online	ϵ -greedy + GLIE
Q-learning	free	off	online	max 算子
DQN	free	off	online	上面三件套 (replay + target net + Double)
SAC	free	off	online	+ 最大熵 + 双 Critic
PPO	free	on	online	信赖域 / clipping
CQL / IQL / BCQ	free	off	offline	Q 值正则 / 避免 max / 行为约束

- model-based vs model-free（要不要P,R）——前几节的主线
- on-policy vs off-policy（执行的策略= 评估的策略吗）
- online vs offline（边交互边学vs 只用历史数据）

on/off-policy的比较：注意关键词“policy”

算法	π_b	π_t	on/off-policy
MC 评估	π	π	on-policy
MC control	ϵ -greedy	ϵ -greedy	on-policy
SARSA	ϵ -greedy	ϵ -greedy	on-policy
Q-learning	ϵ -greedy	greedy	off-policy
UCB-VI	乐观 greedy	乐观 greedy	on-policy

Off-policy 的好处：

1. 探索与利用解耦：用充分探索的 π_b 产生数据，学到完全greedy 的 π_t 。最优行为不被探索噪声污染。
2. 数据复用：旧数据（不管由哪个旧策略产生）可以反复利用——这是经验回放的前提
3. 演示/示教学习：可以从人类专家或别的策略的数据中学习
4. 多策略并行评估：用同一份数据同时评估多个候选target policy

Off-policy 的代价

1. 分布不匹配： π_b 产生的状态/动作分布 $d^{\pi_b} \neq d^{\pi_t}$ ，朴素估计会有偏。修正工具：重要性采样
2. Bellman 最优算子的max 带来过估计（Double Q-learning）
3. 和函数逼近+ bootstrap 组合= Deadly Triad

为了处理off-policy分布不匹配问题，可以使用**重要性采样（importance sampling, IS）**，从 π_b 产生的样本估计 π_t 下的期望。

其实就是用策略的比例给每个样本加权，被 π_b 多观察到的事件权重降低，反之升高。

$$\mathbb{E}_{a \sim \pi_t}[f(a)] = \mathbb{E}_{a \sim \pi_b}\left[\frac{\pi_t(a)}{\pi_b(a)} f(a)\right] = \mathbb{E}_{a \sim \pi_b}[\rho(a) f(a)]$$

对 T 步轨迹做off-policy评估需要乘累积似然比

$$\rho_{0:T-1} = \prod_{t=0}^{T-1} \frac{\pi_t(a_t|s_t)}{\pi_b(a_t|s_t)}$$

方差会指数级爆炸。所以纯IS只在短的horizon情况下可行。

借一个例子回顾一下从Q-learning到TD的根本级区别：多了一个 a_{t+1} ，变成了五元组。Q-learning时用的是四元组： (s_t, a_t, r_t, s_{t+1}) ，不需要 a_{t+1} ，所以不需要一个策略来预测下一个动作。因此，它评估的不是某个目标策略 π_t 的长期价值 Q^{π_t} （“轨迹”是依赖于策略的），而是最优算子 T^* 的不动点 Q^* （“算子”是关于环境的，与策略无关）。因此，Q-learning不需要IS这样的似然比，也就绕过了IS的方差爆炸。

online/offline

online：算法在与环境的持续交互中学习。
offline（又名batch RL）：只有预先收集好的数据集 $D = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^N$ ，不能再与环境交互。

	Online	Offline
On-policy	SARSA, A2C, PPO	不存在 (offline 下 π_b 固定无法 $= \pi_t$)
Off-policy	Q-learning, DQN, SAC	CQL, IQL, BCQ, Decision Transformer

经验回放 (experience replay)：让online算法重复消化样本，只能用于off-policy。

TD/Q-learning 的”原生”形式：观察一个 (s, a, r, s') ，更新一次，丢掉，再观察下一个。
两个浪费：

- 1. 样本只用一次就丢——但收集样本很贵（特别是真实机器人/医疗/工业过程）
- 2. 相邻样本强相关（同一轨迹里前后步状态相似），违反随机逼近定理喜欢的i.i.d.（独立同分布）假设，让神经网络训练不稳定。

经验回放把每个观察到的 (s, a, r, s') 存进一个固定大小的buffer，更新时从中随机抽一个mini-batch。一颗样本被反复消化。

好处是，可以复用一些样本，更好地近似i.i.d，并且mini-batch平均可以降低梯度方差；但是**经验回放只能用于off-policy**，因为数据来自过去的策略，跟现在的policy对不上。

Algorithm 6 Q-learning with Experience Replay

```
1: 初始化  $Q$ , replay buffer  $\mathcal{B} = \emptyset$  (容量  $N$ , 典型  $N = 10^6$ )
2: for 每步  $t$  do
3:   在  $s_t$  用  $\epsilon$ -greedy 选  $a_t$ , 执行后观察  $r_t, s_{t+1}$ 
4:    $\mathcal{B} \leftarrow \mathcal{B} \cup \{(s_t, a_t, r_t, s_{t+1})\}$  (满了 FIFO 丢老的)
5:   从  $\mathcal{B}$  中均匀采样一个 mini-batch  $B$  (典型  $|B| = 32 \sim 256$ )
6:   for  $(s, a, r, s') \in B$  do
7:      $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ 
8:   end for
9: end for
```

有个优化版本叫PER (Prioritized Experience Replay), 给各个样本加个重要性采样权重。但是需要维护sum-tree。

目标网络

Q-learning的target是 $r + \gamma \max_{a'} Q(s', a')$, target里的 Q 和被更新的 Q 是同一个。表格情况下每步只修改一个 (s, a) 影响不大。但是在神经网络中, 一次梯度step同时影响所有 (s, a) 处的 Q 值, 使target大幅波动、bootstrap的偏差被放大, 训练容易发散。目标网络就是维护一个 Q 的滞后副本 Q_{target} 用于target。

如带target network的Q-learning更新:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q_{target}(s', a') - Q(s, a)]$$

我们维护的副本有硬更新和软更新两种方式。

硬更新: 每 C 步 $Q_{target} \leftarrow Q$ 。如DQN。

软更新: 每步 $Q_{target} \leftarrow (1 - \tau)Q_{target} + \tau Q$, 如DDPG、SAC。

不动的target = 转化成了有监督回归。在硬更新的两次同步之间, Q_{target} 固定不动, 所以target $r + \gamma \max Q_{target}(s')$ 对每个样本是个常数——拟合Q 向它逼近就是普通的回归问题, 神经网络擅长。每过C 步换一组新的target, 再回归一段。

类比: 你在调一个PID控制器, 但setpoint一直在跳——调起来当然发散。把setpoint 每段时间冻住, 反而能调进去。

Q-learning存在**过估计偏差**，Q值会被高估。

“平方的期望大于等于期望的平方”

对随机变量 $X_1, \dots, X_n$ ，由于 $\max$ 是凸函数，由Jensen不等式有

$$\mathbb{E}[\max_i X_i] \geq \max_i \mathbb{E}[X_i]$$

等号当且仅当所有 X_i 恒等时成立。

在Q-learning中，考虑 $Q(s', a')$ （带噪声的估计），设 $\mathbb{E}[\epsilon_{a'}] = 0$ ，有

$$Q(s', a') = Q^*(s', a') + \epsilon_{a'},$$

两边先取 a' 的 $\max$ ，再取期望，根据Jensen不等式就得到

$$\mathbb{E}[\max_{a'} Q(s', a')] \geq \max_{a'} Q^*(s', a')$$

即使每个 $\epsilon_{a'}$ 都零均值，被 $\max$ 选中的那个动作的 ϵ 倾向于偏正——因为 $\max$ 偏好挑大噪声的样本。

后果：target 被系统性高估 → Q 被推高 → 下次更新更高 → 一直往上漂，策略对噪声特别敏感，训练有时崩盘。

为了解决这个问题，开发出了**Double Q-learning**方法：

把选动作和评估动作两个独立的 Q 估计。这样的话noise就完成了解耦。

维护两个网络 Q_A, Q_B ，每步随机选一个更新：

$$Q_A(s, a) \leftarrow Q_A(s, a) + \alpha[r + \gamma Q_B(s', \arg\max_{a'} Q_A(s', a')) - Q_A(s, a)]$$

然后再以对称形式更新 Q_B 。

整理一下上一周讲解的**探索策略**：

- ϵ -greedy (MC control, SARSA, Q-learning, DQN)：以 ϵ 概率随机， $1 - \epsilon$ 概率 greedy。简单但不智能（在已经熟悉的状态也乱来）
- UCB 式 bonus (UCB-VI)：把“不确定性”加到价值估计上，引导探索去不熟悉的 (s, a) 。理论好，但需要计数器
- Boltzmann/softmax： $\pi(a|s) \propto \exp(Q(s, a)/T)$ ，温度 T 控制探索强度。比 ϵ -greedy 更平滑，对 Q 差异敏感
- Thompson sampling (贝叶斯 RL)：从 Q 的后验分布采样动作。Bandit 上经常打败 UCB
- 内在动机/curiosity (深度 RL)：用状态预测误差等“惊讶度”作为内在奖励——处理稀疏外部奖励的利器

通用规律：探索越智能（用上更多结构），样本效率越高，但实现复杂度也越高。深度 RL 实际中 ϵ -greedy + replay + target net + Double 仍是常见的组合，简单粗暴常用。

从某个状态 s 出发、按某个策略 π 跑一段轨迹（真实或想象的），把过程中的奖励/状态/动作收集起来。这个操作称为**Rollout**。

Rollout长度	Target	算法
1 步	$r + \gamma V(s')$	TD(0), SARSA, Q-learning
n 步	$\sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V(s_{t+n})$	n-step TD, Rainbow DQN
λ -加权	$\sum_n (1 - \lambda) \lambda^{n-1} G_t^{(n)}$	TD(λ), GAE
整段	G_t	MC

偏差-方差关于**rollout长度**的关系：

- 短rollout，bootstrap占比大，偏差大，方差小。
 - 长rollout，真实奖励占比大，偏差小，方差大。
- ps. 额外说明一下，TD(λ)方法是通过 λ 平滑控制rollout长度。

rollout可以用于做数据采集（同步/异步，online RL主流，PPO/DQN/SAC 的actor）、做决策（MCTS）、生成合成数据（imaginary rollout，Dyna/MBPO/Dreamer）、一步策略改进（经典Bertsekas 方法）。

做数据采集包括**同步和异步**两种方法：

- 同步（A2C, PPO）：一批actor 同时rollout 固定步数，然后learner 一次性更新
- 异步（A3C, IMPALA, Ape-X, SEED RL）：actor 持续rollout、learner 持续更新，互不阻塞。需要处理”actor 用的是旧版策略” 的off-policy 校正

reward可以做**标准化和clipping**。副作用是reward clipping 可能扭曲最优策略（10 分和100 分的区别消失了）。当不同动作的奖励差别在量级上很重要时不要clip。

- Running normalization：维护r 的滑动mean / std，用 $(r - \mu)/\sigma$ 喂给网络
- Reward clipping：DQN 在Atari 上用 $\text{sign}(r) \in \{-1, 0, 1\}$ 暴力clip——丢失幅度信息，但保留了” 好/坏/无”
- Return normalization（PPO 等）：归一化 G_t 而非单步 r

梯度裁剪：TD误差可能会出现极端值（outlier），需要做梯度裁剪。

$$\nabla \leftarrow \nabla \cdot \min(1, \frac{c}{\|\nabla\|_2})$$

典型的 c 取0.5到10。是一个deep RL的默认设置。

Dueling架构：很多状态下做哪个动作都差不多，这时只学 $V(s)$ 就够，不需要单独学每个 $Q(s, a)$ 。此时可以分解为 $Q = V + A$ 。

例如Dueling DQN，网络分两个head分别输出 V 和advantage A ：

$$Q(s, a; \theta) = V(s; \theta_v) + \left(A(s, a; \theta_a) - \frac{1}{|A|} \sum_{a'} A(s, a'; \theta_a) \right)$$

减去均值以实现单位化。

该方法在动作数大、很多状态下动作选择不敏感的任务（Atari 大部分游戏）效果显著。

Huber loss用于Q-learning的损失，例如DQN。小误差处像MSE（光滑、梯度精确），大误差处像MAE（梯度有界，鲁棒）。

$$L_K(\delta) = \begin{cases} \frac{1}{2}\delta^2 & |\delta| \leq \kappa \\ \kappa(|\delta| - \frac{\kappa}{2}) & |\delta| > \kappa \end{cases}$$

熵正则化：在loss中加入 $-\beta H(\pi(\cdot|s))$ ，其中 $H(\pi) = -\sum_a \pi(a|s) \log \pi(a|s)$ 是策略熵， β 控制随机性强度。熵正则化鼓励策略保持随机，防止探索停止陷入局部最优。

例如A2C/PPO、SAC。

但是这样的话最优策略不是原MDP的最优策略。

Hindsight Experience Replay (HER)：用于应对稀疏奖励任务（如机械臂抓取、导航等），把失败的轨迹当成达成了另一个目标的成功轨迹。这个还挺符合直觉的。

potential-based reward shaping：基于势函数，把先验知识“哪些状态更接近成功”编码进 Φ ——例如导航任务设 $\Phi(s) = -\|s - s_{goal}\|$ 。

保证最优策略不变，但学习速度可能数量级提升。

$$R'(s, a, s') = R(s, a) + \gamma\Phi(s') - \Phi(s)$$

model-based/model-free、收敛性理论

算法	收敛性	条件	速度/界
Planning (已知模型)			
值迭代	全局	$\gamma < 1$	$O(\gamma^k)$
策略迭代	全局	有限 MDP	有限步
表格 RL			
Q-learning	全局 w.p.1	无限访问	$O(1/k)$
SARSA	全局 w.p.1	GLIE	$O(1/k)$
TD(λ)	全局	On-policy	依赖 λ
UCB-VI	Regret 界	Episodic	$\tilde{O}(\sqrt{H^3 S A K})$
线性函数逼近			
Linear TD	收敛	On-policy	放大 $\frac{1}{\sqrt{1-\gamma^2}}$
Linear Q	可能发散	Off-policy	—
GTD2	收敛	Off-policy	$O(1/k)$
LSTD	闭式解	批量	$O(\sqrt{d/n})$
Multi-Agent			
Minimax-Q	纳什均衡	两人零和	$O(\gamma^k)$
独立 Q	可能不收敛	一般和	—

$O(\gamma^k)$ ：线性，相对较快。

$O(1/k)$ ：次线性，慢一些。

model-based/model-free

有模型 (model-based) vs 无模型 (model-free) 的核心区别：算法更新价值时，是否直接用到 P 和 R 的显式数学形式。

model-free方法仍需要能在环境中执行动作，观察环境返回的 (r, s') ，收集大量交互样本。offline RL (又称batch RL) 才不需要 P 和 R 的公式、不需要和环境交互、只用预先收集好的历史数据集。

例如，比较一下值迭代 (model-based) 和Q-learning (model-free)。

值迭代的更新公式为

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V_k(s')]$$

(如果公式记得熟，可能会想起来之前写的Bellman最优方程是 $R + \gamma \sum P V_k$ ，而这里求和符号提到最外面了。其实二者是等价的，因为 $\sum_{s'} P(s'|s, a) R(s, a) = R(s, a)$ 的啦)

我们发现这里的值迭代是有模型的，它显式依赖 P 和 R 的解析形式。它不与环境交互，只要给它环境的“规则”，它通过一波推算就能算出最优解。（“同步更新”）

而Q-learning的更新公式为

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

我们发现这里的Q-learning是无模型的，不显式依赖 P 和 R 的解析形式，而是实际执行动作 a 得到环境反馈 s' 和 r 。不需要知道环境的“规则”，但需要能够与环境交互。（“异步更新”）

（第一个收敛：Q-learning收敛性）

接下来给出一个**Q-learning收敛性**的定理（control）。

有趣的是，它和TD(0)的收敛性定理形式几乎完全一致，只是把 $\alpha_t(s)$ 换成了 $\alpha_t(s, a)$ 。

1. 所有 (s, a) 被无限次访问
2. 学习率满足 $\sum_t \alpha_t(s, a) = \infty, \sum_t (\alpha_t(s, a))^2 < \infty$
则 $Q_t \rightarrow Q^*$ 以概率1。

“学习率之和发散”->要持续学习，不能太早停下来

“学习率平方之和收敛”->要逐渐冷静，不能总是大幅调整

例如 $\alpha(s, a) = 1/n$ 就是满足的学习率，其中 n 是 (s, a) 被访问的次数。其本身作为调和级数发散，但是 $1/n^2$ 收敛。

（第二个收敛：TD(0)收敛性）

TD(0)的收敛性定理：

如果所有状态 s 被无限次访问、学习率满足 $\sum_t \alpha_t(s) = \infty, \sum_t \alpha_t(s)^2 < \infty$ ，
则TD(0)以概率1收敛到 V^π 。

例如 $\alpha_t(s) = 1/n$ 就是满足的学习率，其中 n 是 s 被访问的次数。

接下来考虑 $V(s)$ 的线性**函数逼近**。这是牺牲了稳定性，换取可行性的方法。

当 $|S|$ 很大且结构稀疏（如围棋的 10^{170} ），或者状态空间连续（如机器人、自动驾驶等）时，存储完整的 $V(s)$ 表的表格方法不再合适。

解决方法是借助线性函数进行逼近：

$$V_{\theta}(s) = \theta^T \phi(s)$$

其中 θ 是参数， $\phi(s)$ 是状态 s 的特征向量。（注意不是特征值那个特征向量eigenvector，而是feature向量，所以这里其实就是用 Wx 去逼近 V 的意思）

以前我们要对每个 $V(s_t)$ 格子做更新修改。现在 $V(s_t)$ 不再是一个独立变量，而是由 θ 和 $\phi(s_t)$ 计算出来的， θ 是共享的，改变 θ 会同时改变所有状态的 V 值。这样就用了更少的参数 θ ，去逼近了更大的 V 。

“真实的 V 可能不在线性空间里，所以我们在能表示的函数中，找最接近它的那个”

将线性函数逼近代入表格型TD(0)的更新：

$$V(s_t) \leftarrow V(s_t) + \alpha(r_t + \gamma V(s_{t+1}) - V(s_t))$$

得到

$$\theta^T \phi(s_t) \leftarrow \theta^T \phi(s_t) + \alpha(r_t + \gamma \theta^T \phi(s_{t+1}) - \theta^T \phi(s_t))$$

回归一下本质，我们还是为了用 $\theta^T \phi(s_t)$ 逼近TD target: $T_t = r_t + \gamma V(s_{t+1})$ 。

拿 Wx 去逼近另外一个函数，让它们尽量接近，这事儿我们熟啊，梯度下降和最小二乘不就是干这事的吗。定义损失函数

$$L(\theta) = \frac{1}{2} (r_t + \gamma \theta^T \phi(s_{t+1}) - \theta^T \phi(s_t))^2$$

这里 T_t 也依赖于 θ 。如果我们忽略 T_t 对 θ 的依赖（假设目标固定。这个假设还是挺合理的，目标不应该因为参数化而受到影响），对 θ 求梯度得

$$\nabla_{\theta} L(\theta) = -(r_t + \gamma \theta^T \phi(s_{t+1}) - \theta^T \phi(s_t)) \nabla_{\theta} (\theta^T \phi(s_t))$$

又因为 $\nabla_{\theta} (\theta^T \phi(s_t)) = \phi(s_t)$ ，再把这个梯度代入梯度下降的公式

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L$$

就推得了下述的 **Linear TD(0)** 迭代公式：

$$\theta_{t+1} = \theta_t + \alpha_t (r_t + \gamma \theta_t^T \phi(s_{t+1}) - \theta_t^T \phi(s_t)) \phi(s_t)$$

这个结论称作半梯度下降。

接下来介绍它的收敛性。

（第三个收敛：Linear TD(0)收敛性）

在on-policy采样中，**Linear TD(0)**收敛到投影Bellman方程的解：

$$\Phi \theta^* = \Pi_D T^{\pi}(\Phi \theta^*)$$

其中 T^{π} 是Bellman算子， Π_D 是在状态分布 D 加权下到特征空间的投影。

这个逼近的误差界是：

$$\|V_{\theta^*} - V^\pi\|_D \leq \frac{1}{\sqrt{1-\gamma^2}} \min_{\theta} \|V^\pi - \Phi\theta\|_D$$

γ 越大时，线性逼近的误差放大越严重。例如 $\gamma = 0.999$ 时，放大因子 $\frac{1}{\sqrt{1-\gamma^2}}$ 约为22倍。

接下来讲解大名鼎鼎的 **致命三角 Deadly Triad**。

以下三者组合可能导致发散：

1. 函数逼近（参数化，而不是tabular RL）“猜测信息”
2. Bootstrapping（自举，用估计更新估计）“信以为真”
3. Off-policy（行为策略不等于目标策略）“换地方传播”

例如，Baird (1995) 构造了一个7 状态MDP，使用Linear Q-learning（off-policy），发现需要的参数 $\theta \rightarrow \infty$ 。

还好我们有Gradient TD之类的算法。GTD2算法通过显式最小化投影Bellman误差，在off-policy 下也收敛。

避免Deadly Triad可以：

1. 用表格：如果状态空间小，直接用表格，不用函数逼近
 2. 用Monte Carlo：不用bootstrapping，等episode 结束再更新
 3. 用on-policy：只学习你实际执行的策略（如SARSA）
 4. 用GTD：如果必须三者都要，用Gradient TD 类方法
-

接下来讲解Simulation Lemma和Performance Difference Lemma这两个推论。

Simulation Lemma：

设真实MDP为 (P, R) ，学到的模型为 $(\hat{P}, \hat{R})$ ， $\hat{\pi}$ 是学到的模型上的最优策略。则在真实环境中，

$$V^*(s) - V^{\hat{\pi}}(s) \leq \frac{2}{(1-\gamma)^2} \left(\epsilon_R + \gamma \epsilon_P \cdot \frac{R_{max}}{1-\gamma} \right)$$

其中 $\epsilon_R = \max_{s,a} |R - \hat{R}|$ ， $\epsilon_P = \max_{s,a} \|P - \hat{P}\|_1$ 。

一个写绝对值一个写1-范数，是因为 R 和 P 一个是标量一个是向量

例如，取 $\gamma = 0.99, R_{max} = 1, \epsilon_P = \epsilon_R = 0.01$ ，则上述策略次优性上界约为20000。值函数差20000分也太夸张了。所以model-based RL对模型精度要求极高。

model-based RL (MBRL) 有这些痛点：

- 复合误差 (simulation lemma)。
- Vapnik原则。Vapnik 说：“尽量直接解决问题，而不要通过解决一个更难的问题来解决它。”策略往往很简单（“见到悬崖就左转”），但环境动力学极其复杂（悬崖边的风速、地面摩擦力）。非要建模那个复杂环境，属于舍近求远。
- 目标不匹配——预测准不等于拿分高，比如模型能完美生成背景风景画，但完全忽略了致命子弹。Model 预测得准 (Loss 低)，但对RL 任务 (Reward) 没帮助。
- 幻觉作弊，智能体会利用模型的Bug刷分。

为了复活MBRL，因为预测像素（真实物理世界）太难，那就不要预测这么细的东西，做隐空间模型就行了。

MuZero / AlphaZero：

- 不预测棋盘画面，而是预测“价值”和“策略”
- 学会了一个“抽象的规则”
- 在棋类和Atari 游戏中封神

Dreamer (V1/V2/V3)：

- 不预测具体像素，而是在压缩的Latent Space（潜空间）里“做梦”
- 大大减少了复合误差

Yann LeCun：~~我要做World Model!（破音）~~

目前PPO 这种“傻瓜式”Model-Free 算法仍然最好用、最稳定。

Performance Difference Lemma：

$$V^{\pi'}(s) - V^{\pi}(s) = \frac{1}{1 - \gamma} \mathbb{E}_{s' \sim d_s^{\pi'}} [\mathbb{E}_{a \sim \pi'} [A^{\pi}(s', a)]]$$

其中 $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$ 是优势函数。

（回顾一下，之前讲过Dueling架构：很多状态下做哪个动作都差不多，这时只学 $V(s)$ 就够，不需要单独学每个 $Q(s, a)$ 。此时可以分解为 $Q = V + A$ ，其中 A 是advantage。）
我们以前关注值函数，现在主要关注优势函数，注意一下。

优势函数 $A^{\pi}(s, a)$ 说明了如果用旧策略则期望收益是 $V^{\pi}(s)$ ，如果这次选动作 a 则期望收益是 $Q^{\pi}(s, a)$ ，那么 $A^{\pi}(s, a)$ 就是这次选 a 比按旧习惯好多少。

Performance Difference Lemma说，新策略的总收益- 旧策略的总收益= 新策略访问的所有状态上，**新策略的动作比旧策略平均好多少**。它用旧策略的 A^{π} 评估新策略 π' 。

因此，如果有 $\mathbb{E}_{a \sim \pi'}[A^\pi(s', a)] \geq 0$ ，则有 $V^{\pi'} \geq V^\pi$ 。

更进一步，如果有新策略 π' 在每个状态的动作都不比旧策略差 ($A^\pi \geq 0$)，则新策略整体更好。

(例如这个 $A^\pi(s, \pi'(s)) \geq 0$ 在贪婪策略中满足，所以策略迭代具有改进作用；之后我们学习的TRPO/PPO也是利用 A^π 构造代理目标，并约束要求 π' 不离 π 太远)

接下来即将讲解Multi Agent的收敛性。为此首先引入纳什均衡、minimax定理。

一个例子是，两个人要选择动作： $\{6pm\text{做饭}, 8pm\text{做饭}\}$ 。如果两人都选6pm或都选8pm，则各得三分；如果一人选6pm、另一人选8pm，则各得5分。一人的最优选择取决于另一人选什么。

在这种情况下，纳什均衡是双方都以50%随机选，期望收益4分，谁都不想单方面改变。

“如果你知道对方选什么，你就知道做什么；在均衡点，谁都不想单方面改变”

n 人随机博弈定义为 $(S, \{A_i\}, P, \{R_i\}, \gamma)$ ，

每个玩家 i 有自己的动作空间 A_i 和奖励函数 R_i 。

$(\pi_1^*, \dots, \pi_n^*)$ 是**纳什均衡**，如果没有玩家能通过单方面改变策略获益。

两人零和随机博弈定义为 $R_1(s, a_1, a_2) = -R_2(s, a_1, a_2)$ ，

即玩家1的最大化同时对应于玩家2的最小化。“赢家通吃”

Minimax定理：对任意**零和**矩阵博弈 M ，

$$\max_{\pi_1 \in \Delta(A_1)} \min_{a_2 \in A_2} \sum_{a_1} \pi_1(a_1) M(a_1, a_2) = \min_{\pi_2 \in \Delta(A_2)} \max_{a_1 \in A_1} \sum_{a_2} \pi_2(a_2) M(a_1, a_2)$$

这个共同值（博弈的值/期望收益）记为 $\text{val}(M)$ 。

（如果现在还不太会读这个式子的含义，可以先往下看。）

在零和博弈中，先宣布自己策略的一方并没有劣势。可以先用一个石头剪刀布例子简要从直觉上把握一下。

收益矩阵（从你的视角，赢 = +1，输 = -1，平 = 0）：

你 \ 对手	石头	剪刀	布
石头	0	+1	-1
剪刀	-1	0	+1
布	+1	-1	0

- 如果你先选：对手会选针对你的动作，你最多保证 0（比如你选石头，对手选布，你输）
 - 如果对手先选：你可以选针对他的动作，你至少保证 0
- Minimax 定理说：最优混合策略（各 1/3，这个策略是下面可以推出来的！）下，你保证期望收益 0，无论对手怎么选。先选后选都一样。
- 我们推推看这个各 1/3 的最优混合策略。设收益矩阵为

$$M = \begin{pmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{pmatrix}$$

求 $\text{val}(M)$ 。设玩家 1 策略 $\pi = (p, q, 1 - p - q)$ 。

对手最优响应是选择使 $\pi^T M e_j$ 最小的 j 。

（这个 $\pi^T M e_j$ 看起来吓人，其实翻译成成人话是： $\pi^T M$ 是行向量，我们用 e_j 可以取得它的第 j 个分量，得到一个标量。）

$$\pi^T M e_1 = q - (1 - p - q) = p + 2q - 1$$

$$\pi^T M e_2 = -p + (1 - p - q) = 1 - 2p - q$$

$$\pi^T M e_3 = p - q$$

要最大化 $\min_j \pi^T M e_j$ ，令三者相等：

$$p + 2q - 1 = 1 - 2p - q = p - q$$

解得 $p = q = 1 - p - q = 1/3$ 。

所以 $\pi^* = (1/3, 1/3, 1/3)$ ， $\text{val}(M) = 1/3 - 1/3 = 0$ 。

纳什均衡：双方都用 $(1/3, 1/3, 1/3)$ ，值为 0（公平博弈）。

接下来顺便说一下 **minimax vs maximin** 这个式子怎么读。

上述剪刀石头布推得最优策略，从玩家 1 的视角写出的式子就是 maximin：

$$\max_{\pi \in \Delta(A_1)} \min_{j \in A_2} \pi^T M e_j$$

这个式子的含义是玩家 1 先选混合策略 π ，玩家 2 后选纯策略 j （即知道 π 后针对性地最小化玩家 1 的收益）。

（ps. $\Delta(A_1)$ 是指在动作空间 A_1 上的所有概率分布的集合）

注意，它是“**从左往右读**”的， $\max$ 和 $\min$ 并不能任意互换。

先是玩家 1 为了最大化自己的分数选择策略，所以是 $\max_{\pi}$ ；

然后是玩家 2 为了最小化玩家 1 的分数做动作，所以是 $\min_j$ 。

这个事情联系到数学知识就清楚了。在嵌套结构中，

$$\max_x \min_y f(x, y)$$

这个式子是“首先”固定 x （玩家1先选策略），然后再计算 $\min_y f(x, y)$ （用于指导玩家2做动作）。这就是从左往右走，也就是玩家1先选、玩家2后选。

那么再联系一下minimax定理。如果是“从玩家2的视角写出的式子”，就是minimax：

$$\min_{\psi \in \Delta(A_2)} \max_{i \in A_1} e_i^T M \psi$$

这个式子的含义是玩家2想了想玩家1会怎么选、所以选择了最小化玩家1分数的混合策略 ψ ，然后接下来玩家1相对应做出了动作 i 。

（注意，maximin和minimax的区别是“**谁先选了策略**”，相当于谁先给对方暴露了信息，而不是谁先“真的做了动作”。）

minimax定理就是说，对于这样的零和博弈，不管玩家1先选策略、还是玩家2先选策略，得到的期望收益都是完全一样的：

$$\max_{\pi \in \Delta(A_1)} \min_{j \in A_2} \pi^T M e_j = \min_{\psi \in \Delta(A_2)} \max_{i \in A_1} e_i^T M \psi$$

相信说到这个程度，就很清楚为什么能够联系到上面剪刀石头布的“玩家1先选策略”和“玩家2先选策略”的纳什均衡期望都为0了。

（第四个收敛：Minimax-Q的收敛性）

再接下来讲解Multi-Agent RL的收敛性。

当有多个agent同时学习时，每个agent的最优策略依赖于其他agent。

而且其他agent也在学习，环境非平稳。

因此，我们的目标达不到单方最优，而是纳什均衡。

给出Minimax-Q学习更新：

$$Q(s, a_1, a_2) \leftarrow (1 - \alpha)Q(s, a_1, a_2) + \alpha[r + \gamma \text{val}(Q(s', \cdot, \cdot))]$$

可以证明它的收敛性：在满足标准随机逼近条件下， $Q \rightarrow Q^*$ （纳什均衡Q函数）。

这个结论的完整证明类似于压缩映射相关的内容，但不列举完整证明，只提其中关键的一步。

它用到的关键引理是 $\text{val}(\cdot)$ 是非扩张的（non-expansive）：

对任意矩阵 M, M' ，有 $|\text{val}(M) - \text{val}(M')| \leq \|M - M'\|_\infty$ 。简单说明一下证明。

相信根据之前对minimax和maximin的讲解，大家都清楚了“从左往右读”和“谁先选策略”的直觉。设 π_1^* 这个特定的策略是 M 的最优混合策略， π_2^* 是 M' 的最优混合策略。则

$$\text{val}(M) = \max_{\pi_1} \min_{a_2} \pi_1^T M e_{a_2}$$

$\text{val}(M)$ 是最大的那个，所以就是代入最优策略的那个值。（在所有 π_1 对应的 $\text{val}(M)$ 取最大的值，也就是最优的那个 π_1^* 对应的 $\text{val}(M)$ 的值。这个写法下面也会反过来写）

$$\text{val}(M) = \min_{a_2} (\pi_1^*)^T M e_{a_2}$$

又因为

$$(\pi_1^*)^T M e_{a_2} = (\pi_1^*)^T M' e_{a_2} + (\pi_1^*)^T (M' - M) e_{a_2}$$

以及范数的原理（1范数是对每个分量取绝对值再求和，这里的无穷范数是矩阵每行求和再取最大值，1-范数与 ∞ -范数互为对偶、配套使用）

$$(\pi_1^*)^T (M' - M) e_{a_2} \leq \|\pi_1^*\|_1 \|M' - M\|_\infty \|e_{a_2}\|_1$$

把 π_1^* 和 e_{a_2} 这两个向量的各个分量取绝对值再求和，值均为标量1。所以推得

$$\text{val}(M) \leq \min_{a_2} (\pi_1^*)^T M' e_{a_2} + \|M' - M\|_\infty$$

同理前面提到的写法，把最优 π_1^* 反过来写成外层 $\max_{\pi_1}$ 也即

$$\text{val}(M) \leq \max_{\pi_1} \min_{a_2} \pi_1^T M' e_{a_2} + \|M' - M\|_\infty$$

这个形式正是 $\text{val}(M')$ 。所以推出下述式子

$$\text{val}(M) \leq \text{val}(M') + \|M - M'\|_\infty$$

对称地也可推得

$$\text{val}(M') \leq \text{val}(M) + \|M - M'\|_\infty$$

所以有上述的关键引理成立。

我们终于把零和博弈的Minimax-Q收敛性说明清楚了。

可惜，一般和博弈收敛困难，独立Q-learning可能不收敛。

TD(λ)、LSTD、平均奖励MDP

之前的主线：更新算法讲了DP->MC->UCB->TD（SARSA，Q-learning）。

之后一口气讲了很多的指标、工程优化和收敛性理论。

现在继续回到更新算法，把其余的几个算法也讲讲。

TD(λ) 实际上就是为了在偏差和方差之间取得一个平衡，也即在 $TD(0)$ 和 MC 之间平衡。

λ 为0时，等价于 $TD(0)$ ，偏差高、方差低，适用于模型准确的情况、便于快速学习；

λ 为1时，等价于 MC ，偏差低、方差高，适用于episode短、奖励稀疏的情况。

$\lambda = 0.9$ 之类的基本上就能两边平衡一下。

为了实现TD(λ)，有两种视角。

前向视角叫 λ -return，下面介绍，计算量很大。

一种后向视角实现机制叫**资格迹**，和 λ -return等价，是一个工程技巧，适宜于工程实现。

λ -回报 (λ -return) 定义为

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

它是所有 n 步回报 $G_t^{(n)} = \sum_{i=0}^{n-1} \gamma^i r_{t+i} + \gamma^n V(s_{t+n})$ 关于 λ 的指数加权平均。

前向视角的更新目标是 λ -回报 G_t^λ ，

后向视角用资格迹 $e(s) \leftarrow \gamma \lambda e(s) + \mathbf{1}_{s_t=s}$ 。

- 每次访问状态 s ，它的资格迹增加 1
- 每步之后，所有状态的资格迹都乘以 $\gamma \lambda$ （衰减）
资格迹相当于一种“记忆”，记录每个状态最近被访问过多久。

可以证明，上述两种视角是等价的。

在收敛性理论中我们介绍了Linear TD(0)以及半梯度下降。

$$\theta_{t+1} = \theta_t + \alpha_t (r_t + \gamma \theta_t^T \phi(s_{t+1}) - \theta_t^T \phi(s_t)) \phi(s_t)$$

但当时是在线、增量、有学习率的。

用完全相同的最小二乘思想，我们也可以批量、闭式、无学习率做更新，这就是**LSTD**方法。

根据最小二乘原理，LSTD的闭式解是：

$$\theta^* = A^{-1}b$$

$$A = \mathbb{E}[\phi(s)(\phi(s) - \gamma \phi(s'))^T], b = \mathbb{E}[\phi(s)r]$$

我们同样只是在找一个线性函数 $V_\theta(s) = \theta^T \phi(s)$ ，使得它尽可能满足Bellman方程数据点 $(s, r + \gamma V(s'))$ 。

LSTD数据高效、无学习率、off-policy，适用于不能在线交互、特征维度不太高、有一批历史数据的情况。

之前我们一直用到折扣 γ 作为奖励：

$$G_t = \sum_t \gamma^t r_t$$

不过有一些无限期运行的系统使用平均奖励更自然：

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_t r_t$$

因此定义平均奖励

$$\rho^\pi = \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} r_t \right]$$

平均奖励 Bellman 方程

$$\rho^* + h^*(s) = \max_a \left[R(s, a) + \sum_{s'} P(s'|s, a) h^*(s') \right]$$

其中 h^* 是差分价值函数。

这个方法称为 R-learning ，在遍历 MDP 上收敛到 (ρ^*, h^*) 。

完结撒花！